



北京邮电大学
Beijing University of Posts and Telecommunications

北京邮电大学
计算机学院

数据预处理和可视化作业-2

姓名: 吴清柳

学号: 2020211597

班级: 2020211323

指导老师: 王纯

课程名称: *Python* 程序设计

December 8, 2022

Contents

1	概述	3
2	数据处理	3
2.1	概述	3
2.2	输入数据格式	3
2.3	数据预处理	4
2.4	处理线性插值和高度异常数据	5
2.4.1	实现介绍	5
2.4.2	处理结果	5
2.5	处理 PM 指数异常值	6
2.5.1	实现介绍	6
2.5.2	处理结果	7
2.6	处理 cbwd 中值为"cv" 的单元格	7
2.6.1	实现介绍	8
2.6.2	处理结果	8
2.7	归一化处理和散点图表示	8
2.7.1	实现介绍	9
2.7.2	处理结果	9
2.7.3	可视化结果	10
2.8	空气质量数据离散化和饼图表示	11
2.8.1	空气质量分级标准	11
2.8.2	实现介绍	11
2.8.3	处理结果	13
2.8.4	可视化结果	14
3	程序的结构和运行	14
3.1	代码结构	14
3.2	代码运行	17
4	附录	17

1 概述

借助 Pandas 库, 对 [PM2.5 Data of Five Chinese Cities | Kaggle](#) 中的北京空气质量数据进行处理, 包括线性插值处理, 处理高度异常数据, 缺失数据填充等; 并将处理后的数据进行可视化和保存到 csv 文件中.

2 数据处理

2.1 概述

本节各部分分别介绍数据预处理, 以及对数据进行的五项数据处理, 并在部分章节中对数据进行了数据可视化. 所有的代码都在 `src/data_process.py` 中的类 `DataProcess` 中.

1. 对 HUMI, PRES, TEMP 三列, 进行线性插值处理. 并对其中超过 3 倍标准差的高度异常数据, 修改为 3 倍标准差的数值.
2. 假设 PM 指数最高为 500, 对 PM_Dongsi, PM_Dongsihuan, PM_Nongzhanguan 三列中超过 500 的数据, 修改为 500PM 指数进行异常值的处理.
3. 修改 cbwd 列中值为 "cv" 的单元格, 其值用后项数据填充.
4. 对 DEWP 和 TEMP 两列, 分别进行了 0-1 归一化和 Z-Score 归一化处理, 并将结果使用散点图的形式表示.
5. 将北京的空气质量数据进行离散化, 按照空气质量分级标准, 计算出每个级别 (或颜色值) 对应的天数各有多少, 并将结果以饼图的形式进行可视化.

2.2 输入数据格式

输入数据为 csv 表格形式存储的, 按日期排列的北京空气质量状况数据.

```
No,year,month,day,hour,season,PM_Dongsi,PM_Dongsihuan,PM_Nongzhanguan,PM_US
Post,DEWP,HUMI,PRES,TEMP,cbwd,Iws,precipitation,Iprec
1,2010,1,1,0,4,NA,NA,NA,NA,-21,43,1021,-11,NW,1.79,0,0
2,2010,1,1,1,4,NA,NA,NA,NA,-21,47,1020,-12,NW,4.92,0,0
3,2010,1,1,2,4,NA,NA,NA,NA,-21,43,1019,-11,NW,6.71,0,0
4,2010,1,1,3,4,NA,NA,NA,NA,-21,55,1019,-14,NW,9.84,0,0
5,2010,1,1,4,4,NA,NA,NA,NA,-20,51,1018,-12,NW,12.97,0,0
6,2010,1,1,5,4,NA,NA,NA,NA,-19,47,1017,-10,NW,16.1,0,0
7,2010,1,1,6,4,NA,NA,NA,NA,-19,44,1017,-9,NW,19.23,0,0
8,2010,1,1,7,4,NA,NA,NA,NA,-19,44,1017,-9,NW,21.02,0,0
9,2010,1,1,8,4,NA,NA,NA,NA,-19,44,1017,-9,NW,24.15,0,0
...
```

2.3 数据预处理

下面介绍数据集读入和预处理. 数据集读入和预处理在类 `DataProcessor` 中的函数 `_load_data()` 中. 由于 `pandas` 的 `read_csv()` 可以方便的对读入数据的日期进行处理, 所以使用如下代码进行处理, 将数据的日期作为索引, 并让 `pandas` 自主推断日期格式. 此外, 对数据中的 `NaN` 值进行了规定, 让 `Pandas` 可以正确读取 `NaN` 值, 而不是读成字符串.

```
self.raw_df = dataset
# Read csv using pandas and store data in self.raw_dat and store data in
# self.raw_data.
dataset = pd.read_csv(
    in_file,
    parse_dates={"Date": ["year", "month", "day", "hour"]},
    date_parser=lambda x: datetime.strptime(x, "%Y %m %d %H"),
    infer_datetime_format=True,
    index_col="Date",
    na_values=["NaN", "?"],
)
self.raw_df = dataset
```

输入的数据格式如下:

```
No season PM_Dongsi PM_Dongsihuan ... cbwd Iws precipitation Iprec
...
1 4 NaN NaN ... NW 1.79 0.0 0.0
2 4 NaN NaN ... NW 4.92 0.0 0.0
3 4 NaN NaN ... NW 6.71 0.0 0.0
4 4 NaN NaN ... NW 9.84 0.0 0.0
5 4 NaN NaN ... NW 12.97 0.0 0.0
6 4 NaN NaN ... NW 16.10 0.0 0.0
7 4 NaN NaN ... NW 19.23 0.0 0.0
8 4 NaN NaN ... NW 21.02 0.0 0.0
9 4 NaN NaN ... NW 24.15 0.0 0.0
10 4 NaN NaN ... NW 27.28 0.0 0.0
```

输入数据的统计信息如下:

```
--- Input data statistics:
ss 'pandas.core.frame.DataFrame'>
timeIndex: 52584 entries, 2010-01-01 00:00:00 to 2015-12-31 23:00:00
columns (total 14 columns):
Column Non-Null Count Dtype
-----
No 52584 non-null int64
season 52584 non-null int64
PM_Dongsi 25052 non-null float64
PM_Dongsihuan 20508 non-null float64
PM_Nongzhanguan 24931 non-null float64
PM_US Post 50387 non-null float64
DEWP 52579 non-null float64
HUMI 52245 non-null float64
PRES 52245 non-null float64
TEMP 52579 non-null float64
```

```

cbwd 52579 non-null object
lws 52579 non-null float64
precipitation 52100 non-null float64
lprec 52100 non-null float64
es: float64(11), int64(2), object(1)

```

2.4 处理线性插值和高度异常数据

本节介绍对 HUMI, PRES 和 TEMP 三列进行线性插值处理, 并截断其中超出三倍标准差的数据.

对线性插值和高度异常数据 (超过 3 倍标准差) 的数据的处理, 在类 DataProcess 中的函数 linear_interpolate() 中.

2.4.1 实现介绍

使用 pandas 的函数 interpolate() 对数据进行线性插值处理如下:

```

# Linear interpolate
new_col = column.interpolate(
    method="linear", limit_direction="forward")

```

由于 pandas 是 numpy 的扩展, 所以很好的继承了 numpy 的特性, 例如下面用到的 Boolean Array. 对数据中超出三倍标准差的数据, 通过如下方式截断为三倍标准差:

```

# Process highly anomalous data that exceeds 3 standard deviations.
std = column.std()
mean = column.mean()
new_col[column > mean + 3 * std] = 3 * std + mean
new_col[column < mean - 3 * std] = 3 * std - mean

```

2.4.2 处理结果

使用样例数据进行测试. 测试结果中省略了不相关的列:

```

# Raw data before process.
      No season ... DEWP HUMI PRES TEMP ...
Date
2010-01-01 00:00:00 1 4 ... -21 43.0 1021 -11 ...
2010-01-01 01:00:00 2 4 ... -21 47.0 1020 -12 ...
2010-01-01 02:00:00 3 4 ... -21 43.0 1019 -11 ...
2010-01-01 03:00:00 4 4 ... -21 55.0 1019 -14 ...
2010-01-01 04:00:00 5 4 ... -20 NaN 1018 -12 ...
2010-01-01 05:00:00 6 4 ... -19 47.0 1017 -10 ...
2010-01-01 06:00:00 7 4 ... -19 44.0 1017 -9 ...
2010-01-01 07:00:00 8 4 ... -19 440.0 1017 -9 ...
2010-01-01 08:00:00 9 4 ... -19 44.0 1017 -9 ...

```

```

2010-01-01 09:00:00 10 4 ... -20 37.0 1017 -8 ...

# Run log.
Linear interpolate and process highly anomalous data in HUMI.
Modified 1 cells.
Linear interpolate and process highly anomalous data in PRES.
Modified 0 cells.
Linear interpolate and process highly anomalous data in PRES.
Modified 0 cells.

# Processed data.
No season ... DEWP HUMI PRES TEMP ...
Date
2010-01-01 00:00:00 1 4 ... -21 43.0 1021 -11 ...
2010-01-01 01:00:00 2 4 ... -21 47.0 1020 -12 ...
2010-01-01 02:00:00 3 4 ... -21 43.0 1019 -11 ...
2010-01-01 03:00:00 4 4 ... -21 55.0 1019 -14 ...
2010-01-01 04:00:00 5 4 ... -20 51.0 1018 -12 ...
2010-01-01 05:00:00 6 4 ... -19 47.0 1017 -10 ...
2010-01-01 06:00:00 7 4 ... -19 44.0 1017 -9 ...
2010-01-01 07:00:00 8 4 ... -19 440.0 1017 -9 ...
2010-01-01 08:00:00 9 4 ... -19 44.0 1017 -9 ...
2010-01-01 09:00:00 10 4 ... -20 37.0 1017 -8 ...

```

可以看到 HUMI 中的 NaN 被插值为 51.0.

对完整数据集进行测试, 根据输出日志可以看到, 三列中均有数据被进行过插值或修改到三倍标准差内.

```

Linear interpolate and process highly anomalous data in HUMI.
Modified 339 cells.
Linear interpolate and process highly anomalous data in PRES.
Modified 339 cells.
Linear interpolate and process highly anomalous data in PRES.
Modified 5 cells.

```

2.5 处理 PM 指数异常值

本节介绍对 PM_Dongsi, PM_Dongsihuan 和 PM_Nongzhanguan 三列中 PM 指数超过 500 的异常数据进行处理. 处理方法为将其截断为 500.

2.5.1 实现介绍

类似上一节中提到的方式, 使用 pandas 中 DataFrame 的 Boolean Array 对每一列 col 中的异常数据进行处理:

```

def process(col):
    cp = col.copy()
    print(f"Handled {len([x for x in cp[cp > 500]])} values")
    cp[cp > 500] = 500

    return cp

```

2.5.2 处理结果

对样例数据进行测试, 测试结果中省略了不相关的列:

```
# Raw data before process.
      No season PM_Dongsi PM_Dongsihuan PM_Nongzhanguan ...
Date
2010-01-01 00:00:00 1 4 NaN NaN NaN ...
2010-01-01 01:00:00 2 4 NaN NaN NaN ...
2010-01-01 02:00:00 3 4 NaN NaN NaN ...
2010-01-01 03:00:00 4 4 NaN NaN 858.0 ...
2010-01-01 04:00:00 5 4 NaN 39.0 NaN ...
2010-01-01 05:00:00 6 4 NaN NaN NaN ...
2010-01-01 06:00:00 7 4 NaN 1871.0 NaN ...
2010-01-01 07:00:00 8 4 17.0 NaN NaN ...
2010-01-01 08:00:00 9 4 NaN NaN NaN ...
2010-01-01 09:00:00 10 4 NaN NaN NaN ...

# Run log.
Handle values larger than 500 in PM_Dongsi.
Handled 0 values
Handle values larger than 500 in PM_Dongsihuan.
Handled 1 values
Handle values larger than 500 in PM_Nongzhanguan.
Handled 1 values

# Processed data.
      No season PM_Dongsi PM_Dongsihuan PM_Nongzhanguan ...
Date
2010-01-01 00:00:00 1 4 NaN NaN NaN ...
2010-01-01 01:00:00 2 4 NaN NaN NaN ...
2010-01-01 02:00:00 3 4 NaN NaN NaN ...
2010-01-01 03:00:00 4 4 NaN NaN 500.0 ...
2010-01-01 04:00:00 5 4 NaN 39.0 NaN ...
2010-01-01 05:00:00 6 4 NaN NaN NaN ...
2010-01-01 06:00:00 7 4 NaN 500.0 NaN ...
2010-01-01 07:00:00 8 4 17.0 NaN NaN ...
2010-01-01 08:00:00 9 4 NaN NaN NaN ...
2010-01-01 09:00:00 10 4 NaN NaN NaN ...
```

可以看到, PM_Dongsihuan 列中值为 1871.0 的数值被截断为 500.0; PM_Nongzhanguan 中值为 858.0 的被截断为 500.0. 对不超过 500.0 的值不进行处理.

对完整数据集进行测试, 根据输出日志可以看到, 三列中均有 PM 指数超过 500 的异常数据被进行了截断.

```
Handle values larger than 500 in PM_Dongsi.
Handled 70 values
Handle values larger than 500 in PM_Dongsihuan.
Handled 75 values
Handle values larger than 500 in PM_Nongzhanguan.
Handled 81 values
```

2.6 处理 cbwd 中值为"cv"的单元格

本节介绍对 cbwd 中值为"cv"的单元格使用后项数据填充.

2.6.1 实现介绍

关键代码段如下:

```
for i in range(len(cbwd.index), 1, -1):
    if cbwd[i-1] == 'cv':
        cnt += 1
    cbwd[i-1] = cbwd[i] if cbwd[i-1] == "cv" else cbwd[i-1]
```

由于是从后往前替换, 所以即便是多个相连的 cv, 也能完成替换.

2.6.2 处理结果

使用样例数据进行测试, 测试结果中省略了不相关的列:

```
No season ... cbwd Iws precipitation Iprec

1 4 ... NW 1.79 0 0
2 4 ... NW 4.92 0 0
3 4 ... NW 6.71 0 0
4 4 ... NW 9.84 0 0
5 4 ... NW 12.97 0 0
6 4 ... NW 16.10 0 0
7 4 ... NW 19.23 0 0
8 4 ... cv 21.02 0 0
9 4 ... NW 24.15 0 0
10 4 ... NW 27.28 0 0

e value is 'cv' in column 'cbwd'.
No season ... cbwd Iws precipitation Iprec

1 4 ... NW 1.79 0 0
2 4 ... NW 4.92 0 0
3 4 ... NW 6.71 0 0
4 4 ... NW 9.84 0 0
5 4 ... NW 12.97 0 0
6 4 ... NW 16.10 0 0
7 4 ... NW 19.23 0 0
8 4 ... NW 21.02 0 0
9 4 ... NW 24.15 0 0
10 4 ... NW 27.28 0 0
```

可以看到原本是 cv 的一行被替换为了下一行中的 NW.

2.7 归一化处理和散点图表示

本节对 DEWP 和 TEMP 两列分别使用 Pandas 进行了 0-1 归一化和 Z-Score 归一化处理, 并将结果使用 matplotlib 绘制饼图进行可视化处理.

2.7.1 实现介绍

0-1 正规化, 将原始数据缩放到 [0, 1] 区间内:

$$\frac{x - \min}{\max - \min} \quad (1)$$

但是缺点是当新数据加入的时候, 可能导致最大和最小值发生变化, 从而需要重新计算.

z-Score, 将原始数据转换为标准正态分布:

$$\frac{x - \mu}{\sigma} \quad (2)$$

但是缺点是对原始数据的分布有要求, 即原始数据的分布必须是近似正态分布的.

```
min = df["DEWP"].min()
max = df["DEWP"].max()

# 0-1 normalization.
def norm01(x, min, max): return (x - min) / (max - min)
def norm01_col(col): return [norm01(col[i], min, max) for i in
                             range(len(col))]

df["DEWP"] = norm01_col(df["DEWP"])
print(f"0-1 nomalized column 'DEWP'.")

# Normalize column TEMP with Z-Score
def norm_z(x, mean, std): return (x - mean) / std
mean = df["TEMP"].mean()
std = df["TEMP"].std()
def norm_z_col(col): return [norm_z(col[i], mean, std) for i in
                             range(len(col))]
df["TEMP"] = norm_z_col(df["TEMP"])
print(f"z-score normalized column 'TEMP'.")
```

2.7.2 处理结果

```
No season ... DEWP HUMI PRES TEMP ...

1 4 ... -21 43.0 1021 -11 ...
2 4 ... -21 47.0 1020 -12 ...
3 4 ... -21 43.0 1019 -11 ...
4 4 ... -21 55.0 1019 -14 ...
5 4 ... -20 NaN 1018 -12 ...
6 4 ... -19 47.0 1017 -10 ...
7 4 ... -19 44.0 1017 -9 ...
8 4 ... -19 440.0 1017 -9 ...
9 4 ... -19 44.0 1017 -9 ...
10 4 ... -20 37.0 1017 -8 ...
```

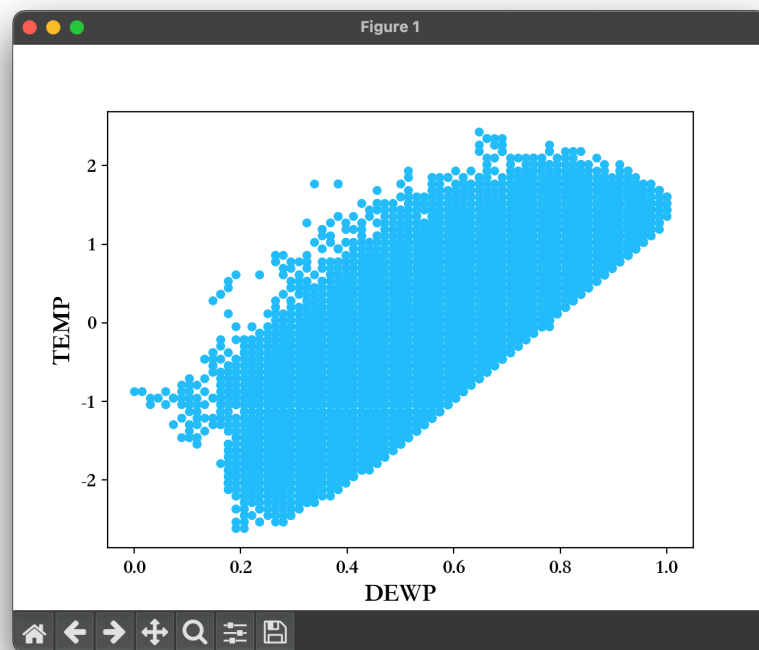


Figure 1: 0-1 正规化和 z-score 正规化散点图

```
'DEWP'.
lumn 'TEMP'.
No season ... DEWP HUMI PRES TEMP ...

1 4 ... 0.0 43.0 1021 -0.271607 ...
2 4 ... 0.0 47.0 1020 -0.814822 ...
3 4 ... 0.0 43.0 1019 -0.271607 ...
4 4 ... 0.0 55.0 1019 -1.901251 ...
5 4 ... 0.5 51.0 1018 -0.814822 ...
6 4 ... 1.0 47.0 1017 0.271607 ...
7 4 ... 1.0 44.0 1017 0.814822 ...
8 4 ... 1.0 440.0 1017 0.814822 ...
9 4 ... 1.0 44.0 1017 0.814822 ...
10 4 ... 0.5 37.0 1017 1.358036 ...
```

可以看到 DEWP 中数据被 0-1 正规化, 而 TEMP 中数据被 z-score 正规化.

2.7.3 可视化结果

由于测试集的数据规模较小, 不能很好的体现正规化后的特征, 因此使用北京 PM 完整数据集对 0-1 正规化和 z-score 正规化的数据使用散点图进行展示如图.

2.8 空气质量数据离散化和饼图表示

本节介绍将北京的空气质量数据进行离散化, 按照空气质量分级表存, 计算出每个级别对应的挺熟, 并将结果以饼图的形式进行可视化. 对每一个 PM 来源均分别进行饼图可视化, 以便于比较.

2.8.1 空气质量分级标准

参考如表 1 空气质量分级标准:

空气质量指数 (AQI)	空气质量指数级别 (状况)
0-50	Excellent(优)
51-100	Good(良)
101-150	Lightly Polluted(轻度污染)
151-200	Moderately Polluted(中度污染)
201-300	Heavily Polluted(重度污染)
300+	Severely Polluted(严重污染)

Table 1: 空气质量分级标准

2.8.2 实现介绍

由于不同观测数据源的 PM 数据有差异, 故使用如下函数对各站点 PM 数据分别进行离散化.

```
# Discretize PM_Dongsi into PM_Dongsi_AQI:
def discretize_pm(column, discretized_column):
    for row in column:
        if 0 <= row <= 50:
            discretized_column.append(aqis.excellent._value_)
        elif 51 <= row <= 100:
            discretized_column.append(aqis.good._value_)
        elif 101 <= row <= 150:
            discretized_column.append(aqis.lightly_polluted._value_)
        elif 151 <= row <= 200:
            discretized_column.append(aqis.moderately_polluted._value_)
        elif 201 <= row <= 300:
            discretized_column.append(aqis.heavily_polluted._value_)
        elif row > 300:
            discretized_column.append(aqis.severely_polluted._value_)
        else:
            # Handle NaN value.
            discretized_column.append(row)
```

数据可视化部分, 首先统计了各空气质量等级的天数:

```

classes = [self.aqicategories.excellent._value_,
self.aqicategories.good._value_,
self.aqicategories.lightly_polluted._value_,
self.aqicategories.moderately_polluted._value_,
self.aqicategories.heavily_polluted._value_,
self.aqicategories.severely_polluted._value_]

def count(col):
    """
    Count AQI categories. Ignore NaN values.
    """
    rslt = [0 for i in range(len(classes))]
    for row in col:
        if row in classes:
            rslt[classes.index(row)] += 1
    return rslt

```

然后对数据绘制饼图进行可视化:

```

data_dongsi = count(df["PM_Dongsi_AQI"])
data_dongsihuan = count(df["PM_Dongsihuan_AQI"])
data_nongzhanguan = count(df["PM_Nongzhanguan_AQI"])
data_us = count(df["PM_US Post_AQI"])

def draw_sub_pie(data, ax_x, ax_y):
    """
    Draw subplot of pie.
    """
    def get_absolute(pct, allvals):
        """
        Get absolute value of days from percentage.
        """
        absolute = int(np.round(pct/100.*np.sum(allvals)))
        return "{:.1f}%\n({:d} days)".format(pct, absolute)

    wedges, texts, autotexts = ax.pie(data, wedgeprops=dict(
        width=0.618), startangle=90, autopct=lambda pct: get_absolute(pct, data))
    bbox_props = dict(boxstyle="square,pad=0.3",
        fc="w", ec="k", lw=0.72)
    kw = dict(arrowprops=dict(arrowstyle="—"),
        bbox=bbox_props, zorder=0, va="center")

    # Compute angle of label line
    for i, p in enumerate(wedges):
        ang = (p.theta2 - p.theta1)/2. + p.theta1
        y = np.sin(np.deg2rad(ang))
        x = np.cos(np.deg2rad(ang))

```

```

horizontalalignment = {-1: "right", 1: "left"}[int(np.sign(x))]
connectionstyle = "angle,angleA=0,angleB={}".format(ang)
kw["arrowprops"].update({"connectionstyle": connectionstyle})
ax.annotate(classes[i] + " " + zh_classes[i], xy=(x, y), xytext=(
    1.35*np.sign(x), 1.2*y), horizontalalignment=horizontalalignment,
    **kw)

# Draw PM_Dongsi pie plot.
fig, ax = plt.subplots(figsize=(
    12, 8), subplot_kw=dict(aspect="equal"))
draw_sub_pie(data_dongsi, 0, 0)
ax.set_title("Dongsi PM AQI")
plt.show()

...

```

2.8.3 处理结果

在样例数据上进行处理得到的结果如下, 由于新增四列后表格太宽, 所以进行了换行:

```

... PM_Dongsi PM_Dongsihuan PM_Nongzhanguan PM_US Post ...
... ..
... NaN NaN NaN NaN ...
... NaN NaN NaN NaN ...
... NaN NaN NaN NaN ...
... NaN NaN 500.0 NaN ...
... NaN 39.0 NaN NaN ...
... NaN NaN NaN 1818.0 ...
... NaN 500.0 NaN 383.0 ...
... 17.0 NaN NaN 12.0 ...
... NaN NaN NaN NaN ...
... NaN NaN NaN NaN ...

PM_Dongsi_AQI PM_Dongsihuan_AQI PM_Nongzhanguan_AQI PM_US Post_AQI

NaN NaN NaN NaN
NaN NaN NaN NaN
NaN NaN NaN NaN
NaN NaN Severely Polluted NaN
NaN Excellent NaN NaN
NaN NaN NaN Severely Polluted
NaN Severely Polluted NaN Severely Polluted
Excellent NaN NaN Excellent
NaN NaN NaN NaN
NaN NaN NaN NaN

```

可以看到对非 NaN 的值, 都进行了正确的对应离散化.

在完整的 Beijing PM 数据集上处理的各观测数据的统计信息如下:

```
PM_Dongsi statistics:
```

```
Excellent[10576]
Good[6268]
Lightly Polluted[3578]
Moderately Polluted[1942]
Heavily Polluted[1910]
Severely Polluted[778]

PM_Dongsihuan statistics:
  Excellent[8104]
  Good[5408]
  Lightly Polluted[3143]
  Moderately Polluted[1683]
  Heavily Polluted[1416]
  Severely Polluted[754]

PM_Nongzhanguan statistics:
  Excellent[10730]
  Good[6259]
  Lightly Polluted[3408]
  Moderately Polluted[1934]
  Heavily Polluted[1697]
  Severely Polluted[903]

PM_US Post statistics:
  Excellent[20050]
  Good[12576]
  Lightly Polluted[7414]
  Moderately Polluted[4274]
  Heavily Polluted[4016]
  Severely Polluted[2057]
```

2.8.4 可视化结果

PM_Dongsi 的可视化结果如图 2.

PM_Dongsihuan 的可视化结果如图 3

PM_Nongzhanguan 的可视化结果如图 4

PM_US Post 的可视化结果如图 5

3 程序的结构和运行

3.1 代码结构

数据处理和数据可视化的代码实现在 src/data_process.py 中的 DataProcessor 类中. 该类主要完成如下三个任务:

数据读取 从 csv 文件中读取北京的 PM 空气质量数据, 并进行预处理.

数据处理 按照作业要求使用 pandas 等工具处理读入的保存在 self.raw_data 中的数据, 将处理后的数据保存在成员 processed_data 中.

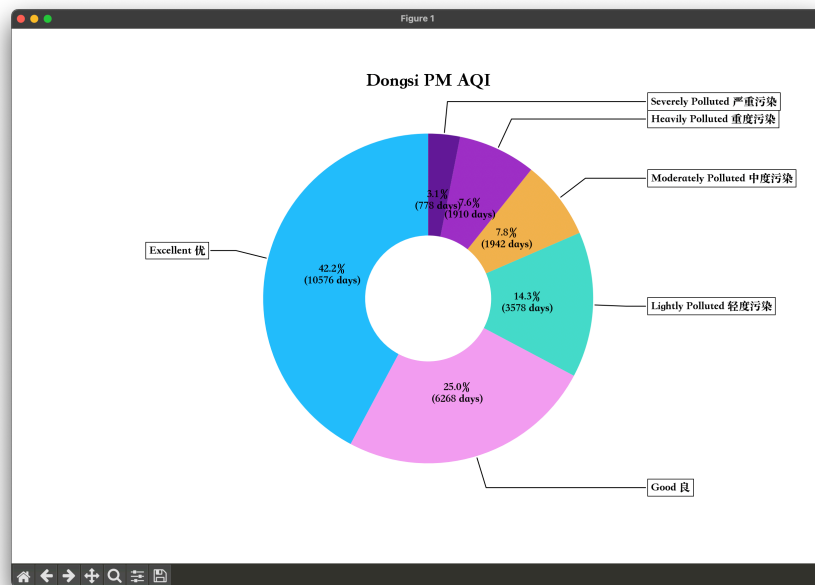


Figure 2: PM_Dongsii 的 AQI 离散化

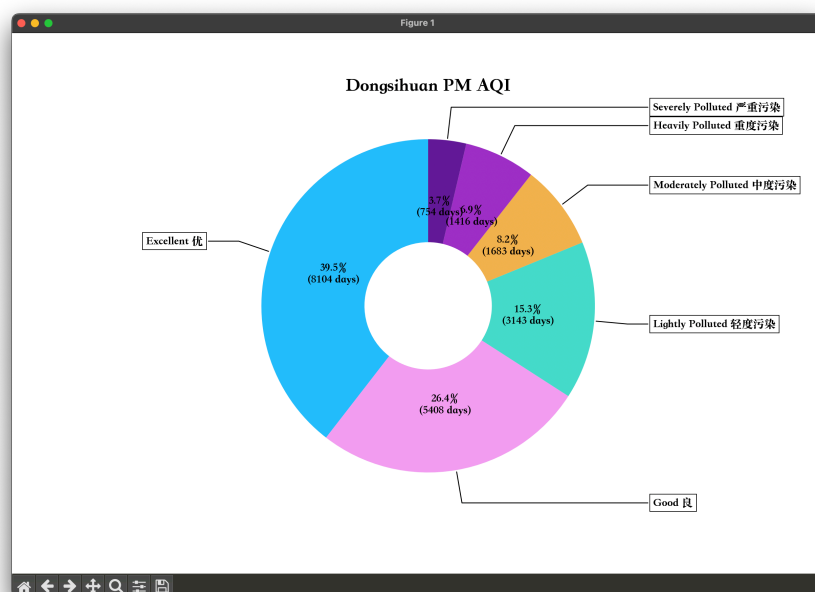


Figure 3: PM_Dongsihuan 的 AQI 离散化

写入 csv 将处理后数据写入 csv 中. 在成员函数 write_data() 实现.

其中, 数据处理部分如下:

线性插值处理 在函数 linear_interpolate() 中.

PM 指数异常值处理 在函数 handle_values_over_500() 中.

后项数据填充 在函数 modify_cbwd() 中.

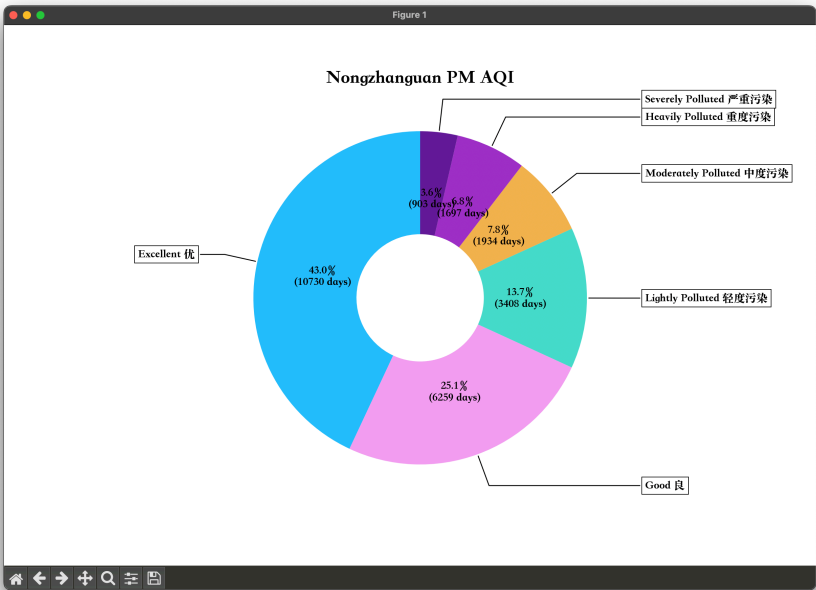


Figure 4: PM_Nongzhanguan 的 AQI 离散化

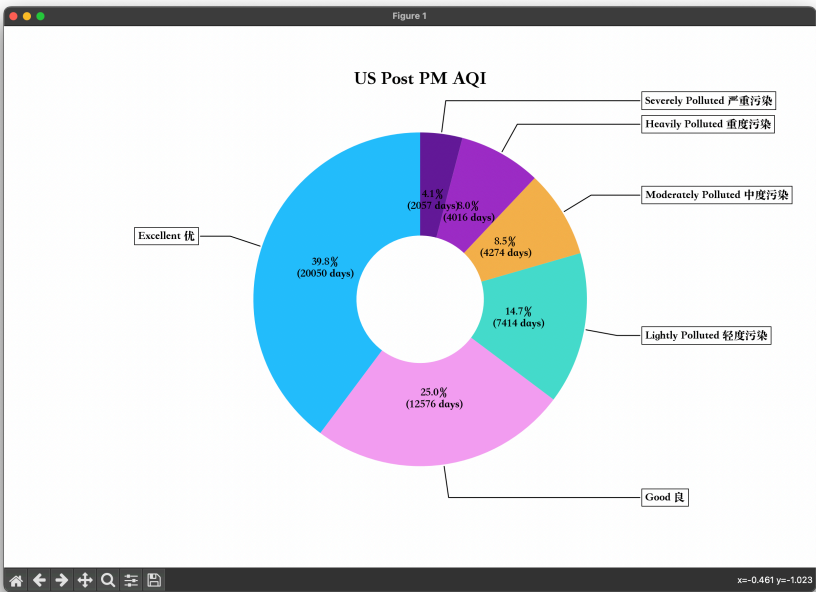


Figure 5: PM_US Post 的 AQI 离散化

归一化处理和散点图表示 在函数 `normalize_process()` 中进行归一化处理, 在函数 `visualize_normalized_data()` 中将归一化处理后的数据以散点图表示.

空气质量数据离散化 在函数 `discretize_aqi()` 中进行空气质量数据离散化, 在函数 `visualize_AQI()` 中将离散化后的空气质量数据以饼图进行可视化.

3.2 代码运行

为脚本添加了命令行参数处理在 main.py 中的类 Parser 中. 使用如下命令可查看帮助:

```
# cwd: src/
$ python data_process.py --help

# output
usage: python main.py [-h] [-i INFILE] [-o OUTFILE]

Scripts to process Beijing PM2.5 data and visulizethe data.

optional arguments:
  -h, --help show this help message and exit
  -i INFILE, --infile INFILE
                        CSV file to process.
  -o OUTFILE, --outfile OUTFILE
                        Path to save processed data.

Author: 2020211323-2020211597-吴清柳
```

除在命令行参数中手动指定输入输出文件外, 还制作了运行脚本 run.sh, 运行如下命令即可读取北京 PM 空气质量数据, 进行数据处理和可视化, 并保存到 csv 文件中.

```
| bash run.sh
```

4 附录

代码仓库: [Github](#)

在作业截止之前内容为空.